BEGINNER → ADVANCED · HANDS-ON GUIDE

The Generative AI & Agentic AI Projects Book

A complete, step-by-step project guide for new developers — build 9 real GenAI and Agentic AI applications from scratch, with full code, architecture diagrams, and explanations.

Python · LLM APIs · Prompt Engineering · RAG · Vector Databases · LangChain · LangGraph · CrewAI · AI Agents

A Practical, Project-Based Learning Path · Edition 2026

Table of Contents

Everything you need, in the order you need it.

PART 01

Foundations

Before you write a single line of AI code, you need the right mental model and the right tools. This part explains what Generative AI and Agentic AI actually are, sets up your environment, and gets your API keys ready — so every project afterward just works.

CHAPTER 1

What is Generative AI & Agentic AI?

Generative AI, in plain terms

Generative AI (GenAI) refers to models — usually Large Language Models (LLMs) like Claude or GPT — that can **generate** new content: text, code, images, audio, and more. You send the model a prompt (an instruction), and it predicts and returns a response. Every project in Part 2 and Part 3 of this book is built on this simple loop: `prompt in → content out`.

Agentic AI, in plain terms

Agentic AI takes this a step further. Instead of just answering once, an **agent** can reason about a goal, decide which **tool** to use (web search, a calculator, a database, your file system), take an action, look at the result, and repeat — in a loop — until the goal is done. This "reason → act → observe" loop is what separates a chatbot from an agent.

This loop repeats until the agent decides the goal is complete.

How this book is structured

Part	What you'll build	Core skill
Part 2	Chatbot, Summarizer, Image Analyzer	Calling LLM APIs, basic prompting
Part 3	PDF Q&A app, Content Generator	RAG, vector databases, prompt templates
Part 4	Research agent, Multi-agent team, Coding agent	Tool use, function calling, multi-agent orchestration
Part 5	Full personal AI assistant	Combining memory + RAG + tools + agents

HOW TO USE THIS BOOK

Do the projects in order. Each one reuses concepts from the last. Type the code yourself instead of copy-pasting — it's the fastest way to actually learn it.

Key vocabulary you'll see throughout this book

Term	Meaning
LLM	Large Language Model — the "brain" that generates text (e.g., Claude, GPT-4)
Prompt	The instruction/text you send to the model
Token	A chunk of text (roughly ¾ of a word) — models are priced and limited by tokens
System Prompt	A hidden instruction that sets the model's role/behavior for the whole conversation
Temperature	A setting (0–1) controlling randomness; low = focused, high = creative
Embedding	A numeric vector representing the "meaning" of text, used for search/similarity
RAG	Retrieval-Augmented Generation — feeding the model relevant retrieved data before it answers
Vector Database	A database optimized to store and search embeddings (e.g., ChromaDB, Pinecone)
Agent	An LLM-driven system that can plan, use tools, and act toward a goal
Function/Tool Calling	A model's ability to output a structured request to run a specific function
Multi-Agent System	Several specialized agents collaborating on one larger task

NOTE

This book uses Python throughout because it has the richest GenAI ecosystem (LangChain, LangGraph, CrewAI, Streamlit). Basic Python knowledge (variables, functions, loops) is the only prerequisite.

CHAPTER 2

Setting Up Your Developer Environment

Set this up once, and every project in this book will run without friction.

1 Install Python 3.10+

Download from `python.org` or use your OS package manager. Verify with:

```
python3 --version
```

2 Install a code editor

VS Code is recommended (free, great Python + Jupyter support). Install the "Python" extension from the marketplace.

3 Create a project folder & virtual environment

A virtual environment keeps each project's dependencies isolated so they don't conflict.

```
mkdir genai-projects && cd genai-projects
python3 -m venv venv

# Activate it:
# macOS/Linux:
source venv/bin/activate
# Windows:
venv\Scripts\activate
```

You'll know it worked when you see `(venv)` at the start of your terminal prompt.

4 Install core libraries

```
pip install anthropic openai python-dotenv streamlit langchain langchain-community langgraph chromadb
pypdf crewai requests
```

Don't worry about memorizing these — each project will tell you exactly which ones it needs.

5 Create a `.env` file for secrets

Never hard-code API keys in your scripts. Create a file named `.env` in your project root:

```
ANTHROPIC_API_KEY=your_key_here
OPENAI_API_KEY=your_key_here
```

> **IMPORTANT**
> Add `.env` to a `.gitignore` file so you never accidentally commit your keys to GitHub.

Recommended folder structure

Use this layout for every project in this book — it keeps things clean and is the same pattern used in professional GenAI codebases.

```
genai-projects/
├── venv/
├── .env
├── .gitignore
├── project-01-chatbot/
│   └── app.py
├── project-02-summarizer/
│   └── app.py
├── project-04-pdf-rag/
│   ├── app.py
│   └── data/
└── requirements.txt
```

Testing your setup

Create a file called `test_setup.py` and run it to confirm everything is wired up correctly before you start Project 1.

```
from dotenv import load_dotenv
import os

load_dotenv()

key = os.getenv("ANTHROPIC_API_KEY")
if key:
    print("✅ API key loaded successfully!")
else:
    print("❌ No API key found. Check your .env file.")
```

```
python test_setup.py
```

> **TIP**
> If this prints the checkmark, you're 100% ready for Chapter 3 and Project 1.

CHAPTER 3

Getting API Keys & Understanding Costs

Option A: Anthropic (Claude)

1 Create an account

Go to `console.anthropic.com` and sign up.

2 Add billing

Add a payment method and a small credit balance (a few dollars is enough for every project in this book).

3 Generate a key

Navigate to **API Keys → Create Key**, copy it immediately, and paste it into your `.env` file.

Option B: OpenAI (GPT)

Same process at `platform.openai.com` — create an account, add billing, generate a key under **API Keys**. Most code in this book works with either provider with a one-line change; we'll note provider-specific code where it matters.

Understanding token-based pricing

LLM APIs charge per token (roughly ¾ of a word), split into **input tokens** (what you send) and **output tokens** (what the model returns), with output usually costing more. A short chatbot conversation typically costs a fraction of a cent; the projects in this book, run a few dozen times each while learning, will typically cost well under $5 total.

Practice	Why it matters
Set a low `max_tokens` while testing	Caps the cost of any single call
Use cheaper/smaller models for development	Save the flagship model for final testing
Set a billing alert/limit in the console	Prevents surprise charges from bugs (like infinite loops)
Cache or store results you'll reuse	Avoid paying to regenerate the same output

ESPECIALLY IMPORTANT FOR PART 4

Agentic projects call the LLM multiple times per task (the reason→act→observe loop). Always set a maximum iteration limit in your agent code so a bug can't create an expensive infinite loop.

You're now fully set up. Let's build your first Generative AI project.

PART 02

Beginner GenAI Projects

Three small, complete projects that teach you the core GenAI loop: sending prompts, handling responses, managing conversation state, and working with images. By the end of this part you'll be comfortable calling an LLM API from any Python script.

PROJECT 1 · BEGINNER

AI Command-Line Chatbot

Time	Difficulty	Stack	You'll learn
45–60 min	⭐ Beginner	Python, Anthropic API	API calls, conversation history, system prompts

What you're building

A terminal chatbot that remembers the conversation, has a customizable personality via a system prompt, and streams responses. This is the "Hello World" of GenAI development — every later project builds on this pattern.

Step-by-step build

1 Create the project folder

```
mkdir project-01-chatbot && cd project-01-chatbot
```

2 Create `chatbot.py` and import dependencies

`chatbot.py`

```
from anthropic import Anthropic
from dotenv import load_dotenv
import os

load_dotenv()
client = Anthropic(api_key=os.getenv("ANTHROPIC_API_KEY"))
MODEL = "claude-sonnet-4-6"
```

3 Define the assistant's personality

The system prompt shapes tone and behavior for the entire chat — change it to build different bots (a tutor, a code reviewer, a career coach).

```
SYSTEM_PROMPT = (
    "You are Nova, a friendly and concise coding tutor. "
    "Explain concepts simply, use short examples, "
    "and always end with one practical suggestion."
)
```

4 Build the conversation loop

The key idea: every message — yours and the model's — gets appended to a `messages` list, and the **whole list** is sent on every turn. This is how the model "remembers" context.

```
messages = []

print("Nova is ready. Type 'exit' to quit.\n")

while True:
```

```
    user_input = input("You: ")
    if user_input.lower() in ("exit", "quit"):
        print("Nova: Goodbye! 👋")
        break

    messages.append({"role": "user", "content": user_input})

    response = client.messages.create(
        model=MODEL,
        max_tokens=500,
        system=SYSTEM_PROMPT,
        messages=messages,
    )

    reply = response.content[0].text
    print(f"Nova: {reply}\n")

    messages.append({"role": "assistant", "content": reply})
```

5 Run it

```
python chatbot.py
```

Try asking a follow-up question ("explain that more simply") without repeating context — it should remember what "that" refers to.

Leveling it up

6 Add streaming responses (feels much more "alive")

```
with client.messages.stream(
    model=MODEL,
    max_tokens=500,
    system=SYSTEM_PROMPT,
    messages=messages,
) as stream:
    print("Nova: ", end="")
    reply = ""
    for text in stream.text_stream:
        print(text, end="", flush=True)
        reply += text
    print("\n")
```

7 Guard against unbounded history (and rising cost)

Long chats mean bigger, pricier requests. Trim old turns once the history grows past a limit:

```
MAX_TURNS = 20
if len(messages) > MAX_TURNS * 2:
    messages = messages[-MAX_TURNS*2:]
```

COMMON BUGS & FIXES

"KeyError / AuthenticationError" → your `.env` key is missing or misspelled.
Bot "forgets" things → check you're appending both user and assistant turns to `messages`.
Responses cut off → raise `max_tokens`.

Challenge yourself

- ☐ Add a command `/save` that writes the conversation to a text file
- ☐ Let the user pick a personality at startup from a menu
- ☐ Add a token/cost counter using `response.usage`

WHAT YOU JUST LEARNED

How to call an LLM, structure conversation history, use a system prompt, and stream output. Every remaining project reuses this exact pattern.

PROJECT 2 · BEGINNER

AI Text Summarizer Web App

Time	Difficulty	Stack
60–90 min	⭐⭐ Beginner+	Python, Streamlit, Anthropic API

You'll learn
Web UIs with Streamlit, prompt design, output formatting

What you're building

A browser-based app where a user pastes long text (or uploads a .txt file) and instantly gets a summary — with a slider to control length and a dropdown to control style (bullet points, executive summary, ELI5). This introduces Streamlit, the fastest way to turn a Python script into a shareable web app.

Step-by-step build

1 Install Streamlit and set up the folder

```
mkdir project-02-summarizer && cd project-02-summarizer
pip install streamlit
```

2 Build the page layout

`app.py`

```
import streamlit as st
from anthropic import Anthropic
from dotenv import load_dotenv
import os

load_dotenv()
client = Anthropic(api_key=os.getenv("ANTHROPIC_API_KEY"))

st.set_page_config(page_title="AI Summarizer", page_icon="📝")
st.title("📝 AI Text Summarizer")
st.caption("Paste text, pick a style, get a summary in seconds.")
```

3 Add inputs: text box, style dropdown, length slider

```
text_input = st.text_area("Paste your text here", height=250)

col1, col2 = st.columns(2)
with col1:
    style = st.selectbox(
        "Summary style",
        ["Bullet points", "Executive summary", "Explain like I'm 5"],
    )
with col2:
    length = st.slider("Target length (sentences)", 1, 10, 4)

go = st.button("Summarize", type="primary")
```

4 Build a dynamic prompt from the user's choices

This is prompt engineering in practice: turn UI selections into precise natural-language instructions.

```
def build_prompt(text, style, length):
    style_instructions = {
        "Bullet points": f"as {length} concise bullet points",
        "Executive summary": f"as a {length}-sentence executive summary",
        "Explain like I'm 5": f"in {length} simple sentences a child could understand",
    }
    return (
        f"Summarize the following text {style_instructions[style]}. "
        f"Only return the summary, no preamble.\n\nTEXT:\n{text}"
    )
```

5 Wire the button to the API call

```
if go:
    if not text_input.strip():
        st.warning("Paste some text first.")
    else:
        with st.spinner("Summarizing..."):
            prompt = build_prompt(text_input, style, length)
            response = client.messages.create(
                model="claude-sonnet-4-6",
                max_tokens=400,
                messages=[{"role": "user", "content": prompt}],
            )
            summary = response.content[0].text
        st.success("Done!")
        st.markdown(summary)
        st.download_button("Download summary", summary, file_name="summary.txt")
```

6 Run the app

```
streamlit run app.py
```

Your browser opens automatically at `localhost:8501`.

Adding file upload support

7 Let users upload a .txt or .pdf instead of pasting

```
uploaded_file = st.file_uploader("...or upload a file", type=["txt", "pdf"])

if uploaded_file:
    if uploaded_file.type == "text/plain":
        text_input = uploaded_file.read().decode("utf-8")
    else:
        import pypdf
        reader = pypdf.PdfReader(uploaded_file)
        text_input = "\n".join(page.extract_text() for page in reader.pages)
```

WATCH OUT FOR LONG DOCUMENTS

Very large files can exceed the model's context window or blow past your token budget. Truncate or chunk text over ~15,000 words before sending it (Project 4 covers chunking properly).

Challenge yourself

- ☐ Add a "compare two summaries side by side" mode
- ☐ Persist history of past summaries using `st.session_state`
- ☐ Add a word-count badge showing compression ratio (original vs. summary)

WHAT YOU JUST LEARNED

How to build a real web UI around an LLM call, turn UI state into a dynamic prompt, and handle file uploads — the foundation for every app in Part 3.

PROJECT 3 · BEGINNER

Multimodal Image Caption & Analysis App

Time	Difficulty	Stack
60 min	⭐⭐ Beginner+	Python, Streamlit, Anthropic Vision API

You'll learn
Sending images to an LLM, multimodal prompts, base64 encoding

What you're building

An app where a user uploads a photo and asks natural-language questions about it: "What's in this image?", "Is this food healthy?", "Extract the text from this receipt." This teaches you multimodal prompting — combining images and text in a single request.

Step-by-step build

1 Set up the folder

```
mkdir project-03-vision && cd project-03-vision
```

2 Build the upload UI

`app.py`

```
import streamlit as st
import base64
from anthropic import Anthropic
from dotenv import load_dotenv
import os

load_dotenv()
client = Anthropic(api_key=os.getenv("ANTHROPIC_API_KEY"))

st.set_page_config(page_title="AI Vision Analyzer", page_icon="🖼")
st.title("🖼 AI Image Analyzer")

uploaded = st.file_uploader("Upload an image", type=["png", "jpg", "jpeg"])
question = st.text_input(
    "What do you want to know about it?",
    value="Describe this image in detail."
)
```

3 Encode the image and send it alongside your question

Images must be base64-encoded and included as a content block, next to a text block, in the same message.

```
if uploaded and st.button("Analyze", type="primary"):
    st.image(uploaded, caption="Your image", use_column_width=True)

    image_bytes = uploaded.getvalue()
    image_b64 = base64.b64encode(image_bytes).decode("utf-8")
    media_type = uploaded.type  # e.g. "image/png"

    with st.spinner("Analyzing..."):
        response = client.messages.create(
```

```
            model="claude-sonnet-4-6",
            max_tokens=500,
            messages=[{
                "role": "user",
                "content": [
                    {
                        "type": "image",
                        "source": {
                            "type": "base64",
                            "media_type": media_type,
                            "data": image_b64,
                        },
                    },
                    {"type": "text", "text": question},
                ],
            }],
        )

    st.markdown("### Result")
    st.write(response.content[0].text)
```

4 Run it

```
streamlit run app.py
```

Practical use-case presets

Turn this into a genuinely useful tool by offering preset prompts instead of a blank text box:

```
preset = st.selectbox("Or pick a use case", [
    "Custom question",
    "Extract all text (OCR)",
    "Describe for accessibility (alt text)",
    "Identify objects and count them",
    "Critique this photo's composition",
])

presets = {
    "Extract all text (OCR)": "Extract every piece of text visible in this image, preserving structure.",
    "Describe for accessibility (alt text)": "Write a concise, descriptive alt-text caption for a screen
reader.",
    "Identify objects and count them": "List every distinct object you see and how many of each.",
    "Critique this photo's composition": "Critique this photo's composition, lighting, and framing like a
photography teacher.",
}
if preset != "Custom question":
    question = presets[preset]
```

Challenge yourself

- ☐ Support multiple images in one request (e.g., "spot the difference")
- ☐ Add a "compare two products" mode with two upload slots
- ☐ Cache results by image hash so re-analyzing the same photo is free

WHAT YOU JUST LEARNED

How to send images to an LLM as structured content blocks. This same technique — mixing content types in one message — is exactly how tools and documents get passed to agents in Part 4.

PART 03

Intermediate: RAG & Prompt Engineering

LLMs don't know your documents or your company's data. Retrieval-Augmented Generation (RAG) fixes that by retrieving relevant information and feeding it into the prompt. This part also covers structured prompt templates for reliable, reusable content generation.

PROJECT 4 · INTERMEDIATE

Chat With Your PDF (RAG App)

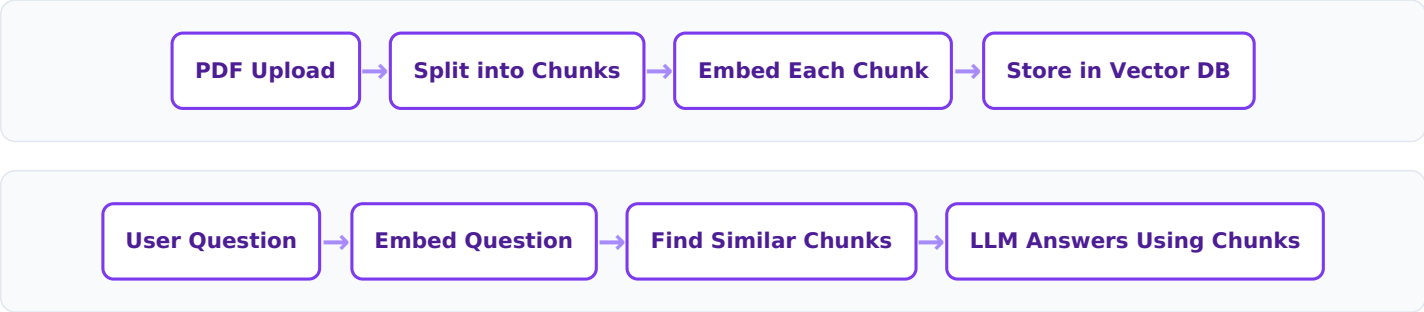
Time	Difficulty	Stack
2–3 hrs	⋆⋆⋆ Intermediate	Python, LangChain, ChromaDB, Streamlit

You'll learn
Chunking, embeddings, vector search, RAG pipelines

What you're building

An app where you upload a PDF (a textbook chapter, a manual, a contract) and ask it questions in plain English, with answers grounded in the actual document — including which section the answer came from.

How RAG works, visually



The key insight: instead of sending the whole PDF to the model every time (expensive, and may not even fit), you only send the **most relevant few paragraphs**, found via similarity search over embeddings.

Step-by-step build

1 Install dependencies

```
mkdir project-04-pdf-rag && cd project-04-pdf-rag
pip install langchain langchain-community langchain-anthropic chromadb pypdf streamlit sentence-transformers
```

2 Load and split the PDF into chunks

Chunking matters: too large and retrieval is imprecise; too small and you lose context. ~800 characters with overlap is a solid default.

`ingest.py`

```
from langchain_community.document_loaders import PyPDFLoader
from langchain.text_splitter import RecursiveCharacterTextSplitter

def load_and_split(pdf_path):
    loader = PyPDFLoader(pdf_path)
    pages = loader.load()

    splitter = RecursiveCharacterTextSplitter(
        chunk_size=800,
        chunk_overlap=100,
    )
    chunks = splitter.split_documents(pages)
```

```
    return chunks
```

3 Embed the chunks and store them in ChromaDB

```
from langchain_community.embeddings import HuggingFaceEmbeddings
from langchain_community.vectorstores import Chroma

def build_vector_store(chunks, persist_dir="chroma_db"):
    embeddings = HuggingFaceEmbeddings(model_name="all-MiniLM-L6-v2")
    vectordb = Chroma.from_documents(
        documents=chunks,
        embedding=embeddings,
        persist_directory=persist_dir,
    )
    return vectordb
```

WHY A LOCAL EMBEDDING MODEL?

Using a free local model (`all-MiniLM-L6-v2`) for embeddings avoids API costs for every chunk — you only pay for the LLM call that generates the final answer.

4 Retrieve relevant chunks for a question

`rag.py`

```
def retrieve_context(vectordb, question, k=4):
    results = vectordb.similarity_search(question, k=k)
    context = "\n\n---\n\n".join(
        f"[Page {doc.metadata.get('page', '?')}]\n{doc.page_content}"
        for doc in results
    )
    return context
```

5 Build the RAG prompt and generate the answer

Explicitly instruct the model to answer **only** from the provided context — this is the single most important line for reducing hallucination.

```
from anthropic import Anthropic
import os

client = Anthropic(api_key=os.getenv("ANTHROPIC_API_KEY"))

def answer_question(vectordb, question):
    context = retrieve_context(vectordb, question)

    prompt = f"""Answer the question using ONLY the context below.
If the answer isn't in the context, say "I couldn't find that in the document."
Cite the page number(s) you used.

CONTEXT:
{context}

QUESTION: {question}"""

    response = client.messages.create(
        model="claude-sonnet-4-6",
        max_tokens=500,
        messages=[{"role": "user", "content": prompt}],
    )
    return response.content[0].text
```

6 Wrap it in a Streamlit UI

`app.py`

```
import streamlit as st
from ingest import load_and_split, build_vector_store
from rag import answer_question

st.title("📄 Chat With Your PDF")

uploaded = st.file_uploader("Upload a PDF", type="pdf")
if uploaded and "vectordb" not in st.session_state:
    with open("temp.pdf", "wb") as f:
        f.write(uploaded.read())
    with st.spinner("Reading and indexing your PDF..."):
        chunks = load_and_split("temp.pdf")
        st.session_state.vectordb = build_vector_store(chunks)
    st.success(f"Indexed {len(chunks)} chunks. Ask away!")

if "vectordb" in st.session_state:
```

```
    question = st.text_input("Ask a question about the PDF")
    if question:
        with st.spinner("Thinking..."):
            answer = answer_question(st.session_state.vectordb, question)
        st.markdown(answer)
```

Improving retrieval quality

Technique	What it fixes
Increase `k` (chunks retrieved)	Answer missing information that was split across chunks
Add metadata filtering (by page/section)	Users asking about a specific chapter
Hybrid search (keyword + vector)	Exact terms (names, codes) that embeddings miss
Re-ranking retrieved chunks with an LLM	Irrelevant chunks that are semantically "close" but not useful

COMMON PITFALL

If answers seem to ignore the PDF, print `context` before sending it to the model — 90% of the time the retrieved chunks are wrong, not the model.

Challenge yourself

- ☐ Support multiple PDFs in one session, with source-document filtering
- ☐ Add a "show sources" expander showing exactly which chunks were used
- ☐ Persist the vector store to disk so re-uploading isn't needed each run

WHAT YOU JUST LEARNED

The full RAG pipeline: chunking, embedding, vector storage, retrieval, and grounded generation. This exact pattern powers Project 9's document memory.

PROJECT 5 · INTERMEDIATE

AI Blog & Content Generator

Time	Difficulty	Stack
90 min	⋆⋆⋆ Intermediate	Python, LangChain prompt templates, Streamlit

You'll learn
Reusable prompt templates, multi-step generation chains, structured output

What you're building

A tool that takes a topic and produces a full blog post: title options, outline, full draft, and an SEO meta description — generated as a **chain** of prompts where each step's output feeds the next, instead of one giant unreliable prompt.

Step-by-step build

1 Set up the project

```
mkdir project-05-content-gen && cd project-05-content-gen
```

2 Write a reusable prompt-template helper

Instead of hand-building prompt strings everywhere, wrap them in template functions — this is what "prompt engineering at scale" looks like.

`prompts.py`

```
OUTLINE_TEMPLATE = """You are an expert content strategist.
Create a blog post outline for the topic: "{topic}"
Audience: {audience}
Return 4-6 section headers only, one per line, no extra text."""

SECTION_TEMPLATE = """Write the "{section}" section of a blog post about "{topic}".
Audience: {audience}. Tone: {tone}.
Write 2-3 paragraphs. Do not repeat the section header."""

META_TEMPLATE = """Write an SEO meta description (max 155 characters)
for a blog post titled about "{topic}"."""
```

3 Build the generation chain

`generate.py`

```
from anthropic import Anthropic
from prompts import OUTLINE_TEMPLATE, SECTION_TEMPLATE, META_TEMPLATE
import os

client = Anthropic(api_key=os.getenv("ANTHROPIC_API_KEY"))

def call(prompt, max_tokens=600):
```

```
    response = client.messages.create(
        model="claude-sonnet-4-6",
        max_tokens=max_tokens,
        messages=[{"role": "user", "content": prompt}],
    )
    return response.content[0].text

def generate_post(topic, audience="general readers", tone="friendly and informative"):
    # Step 1: outline
    outline_text = call(OUTLINE_TEMPLATE.format(topic=topic, audience=audience))
    sections = [s.strip("- ").strip() for s in outline_text.split("\n") if s.strip()]

    # Step 2: expand each section
    body_parts = []
    for section in sections:
        content = call(SECTION_TEMPLATE.format(
            section=section, topic=topic, audience=audience, tone=tone
        ))
        body_parts.append(f"## {section}\n\n{content}")

    # Step 3: SEO meta description
    meta = call(META_TEMPLATE.format(topic=topic), max_tokens=100)

    full_post = "\n\n".join(body_parts)
    return {"outline": sections, "post": full_post, "meta": meta}
```

4 Build the Streamlit interface

`app.py`

```
import streamlit as st
from generate import generate_post

st.title("✍️ AI Blog Generator")

topic = st.text_input("Blog topic", placeholder="e.g. Why beginners should learn Python first")
audience = st.text_input("Audience", value="new developers")
tone = st.selectbox("Tone", ["friendly and informative", "professional", "witty and casual"])

if st.button("Generate Post", type="primary") and topic:
    with st.spinner("Writing your post... this takes a few steps"):
        result = generate_post(topic, audience, tone)

    st.subheader("Outline")
    st.write(result["outline"])

    st.subheader("Full Post")
    st.markdown(result["post"])

    st.subheader("SEO Meta Description")
    st.code(result["meta"])

    st.download_button("Download as Markdown", result["post"], file_name="post.md")
```

Why chaining beats one big prompt

One giant prompt	Chained prompts (this project)
Model may skip or shortchange sections	Every section gets full attention and token budget
Hard to reuse just one piece (e.g., regenerate one section)	Each step is independently re-runnable
Fixed structure baked into one prompt	Easy to insert new steps (e.g., add a fact-check step)

Challenge yourself

- ☐ Add a "regenerate this section only" button per section
- ☐ Add a fact-check pass that flags claims needing a source
- ☐ Support other formats: LinkedIn post, product description, email newsletter

WHAT YOU JUST LEARNED

Multi-step prompt chains — breaking one big task into smaller, controllable LLM calls. This is the conceptual bridge into agents, which decide their own steps dynamically instead of following a fixed chain.

PART 04

Agentic AI Projects

This is where things get exciting. You'll build systems that don't just answer once — they plan, choose tools, take actions, and iterate until a goal is achieved. This is the skillset behind today's AI coding assistants, research agents, and autonomous workflows.

PROJECT 6 · AGENTIC

Autonomous Research Agent (Tool-Using)

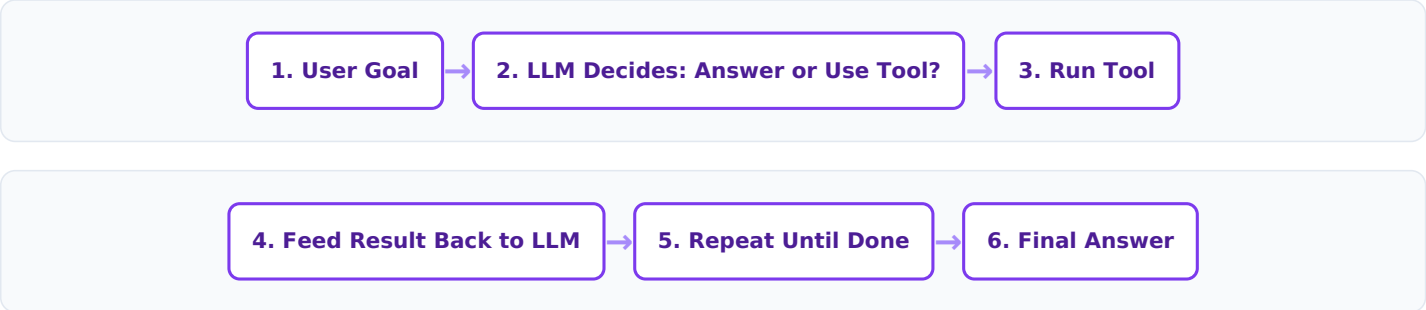
Time	Difficulty	Stack
2–3 hrs	⋆⋆⋆⋆ Advanced	Python, Anthropic tool use, web search API

You'll learn
Tool/function calling, the agent loop, stopping conditions

What you're building

An agent that takes a research question, decides it needs to search the web, calls a search tool, reads results, decides if it has enough information, and — if not — searches again, before finally producing a cited answer. This is your first real agent.

The agent loop, in detail



Step-by-step build

1 Set up and get a search API key

Sign up for a free tier at a search API provider (e.g., Tavily, at `tavily.com`, is built for AI agents). Add `TAVILY_API_KEY` to your `.env`.

```
mkdir project-06-research-agent && cd project-06-research-agent
pip install anthropic tavily-python python-dotenv
```

2 Define the tool the agent can use

A "tool" has two parts: (a) a JSON schema describing it to the model, and (b) the actual Python function that runs when called.

`tools.py`

```
from tavily import TavilyClient
import os

tavily = TavilyClient(api_key=os.getenv("TAVILY_API_KEY"))

TOOLS_SCHEMA = [
    {
        "name": "web_search",
        "description": "Search the web for current information on a topic.",
        "input_schema": {
            "type": "object",
            "properties": {
                "query": {"type": "string", "description": "The search query"}
```

```
            },
            "required": ["query"],
        },
    }
]

def web_search(query):
    results = tavily.search(query=query, max_results=4)
    return "\n\n".join(
        f"{r['title']}\n{r['content'][:400]}\nSource: {r['url']}"
        for r in results["results"]
    )

def run_tool(name, tool_input):
    if name == "web_search":
        return web_search(tool_input["query"])
    return f"Unknown tool: {name}"
```

3 Build the agent loop

The model responds with `stop_reason == "tool_use"` when it wants to call a tool. You run the tool, append the result, and call the model again — repeating until it responds with a final answer.

`agent.py`

```
from anthropic import Anthropic
from tools import TOOLS_SCHEMA, run_tool
import os

client = Anthropic(api_key=os.getenv("ANTHROPIC_API_KEY"))
MODEL = "claude-sonnet-4-6"
MAX_ITERATIONS = 6

SYSTEM_PROMPT = (
    "You are a careful research assistant. Use the web_search tool "
    "when you need current or factual information. Once you have enough "
    "information, give a clear, well-cited final answer. Do not loop "
    "unnecessarily."
)

def run_agent(question):
    messages = [{"role": "user", "content": question}]

    for i in range(MAX_ITERATIONS):
        response = client.messages.create(
            model=MODEL,
            max_tokens=1000,
            system=SYSTEM_PROMPT,
            tools=TOOLS_SCHEMA,
            messages=messages,
        )

        if response.stop_reason != "tool_use":
            final_text = "".join(
                block.text for block in response.content if block.type == "text"
            )
            return final_text

        messages.append({"role": "assistant", "content": response.content})

        tool_results = []
        for block in response.content:
            if block.type == "tool_use":
                print(f"🔧 Agent is calling: {block.name}({block.input})")
                result = run_tool(block.name, block.input)
                tool_results.append({
                    "type": "tool_result",
                    "tool_use_id": block.id,
                    "content": result,
                })
        messages.append({"role": "user", "content": tool_results})

    return "Reached max iterations without a final answer."
```

4 Run it

`main.py`

```
from agent import run_agent
```

```
question = input("Research question: ")
answer = run_agent(question)
print("\n=== FINAL ANSWER ===\n")
print(answer)
```

```
python main.py
# Try: "What are the most notable AI model releases in the last month?"
```

Why the max-iterations guard matters

Without `MAX_ITERATIONS`, a confused agent could search endlessly, generating cost with every loop. This single safeguard is the difference between a toy demo and production-safe agent code — never ship an agent loop without one.

Adding a second tool: a calculator

Agents get more useful with more tools. Add this to `tools.py` to see the model choose between tools automatically based on the question:

```
{
    "name": "calculator",
    "description": "Evaluate a mathematical expression.",
    "input_schema": {
        "type": "object",
        "properties": {"expression": {"type": "string"}},
        "required": ["expression"],
    },
}

# in run_tool():
if name == "calculator":
    return str(eval(tool_input["expression"]))  # use a safe parser in production
```

SECURITY NOTE

`eval()` is used here only for teaching simplicity. In any real project, use a safe math parser like `numexpr` or `asteval` instead — never `eval` untrusted input.

Challenge yourself

- ☐ Add a "save to file" tool so the agent can write its findings to disk
- ☐ Log every tool call to a sidebar so users can see the agent's reasoning trail
- ☐ Add a tool the agent must ask permission before using (human-in-the-loop)

WHAT YOU JUST LEARNED

The complete tool-use agent loop — the foundational pattern behind every agent framework (LangGraph, CrewAI, AutoGPT). Everything from here builds on this loop.

PROJECT 7 · AGENTIC

Multi-Agent Content Team (CrewAI)

Time	Difficulty	Stack	You'll learn
2–3 hrs	⋆⋆⋆⋆ Advanced	Python, CrewAI	Multi-agent orchestration, role specialization, task handoffs

What you're building

A "crew" of three specialized agents — a Researcher, a Writer, and an Editor — that collaborate to turn a topic into a polished article, each agent handing its output to the next. This models how real teams work, and how production multi-agent systems are structured.

Why multiple agents instead of one?

A single agent juggling "research + write + edit" in one prompt tends to blend roles and lose quality. Splitting into specialized agents — each with a narrow role, its own instructions, and sometimes its own tools — produces sharper output and is easier to debug, because you can inspect exactly what each agent produced.

Step-by-step build

1 Install CrewAI

```
mkdir project-07-agent-crew && cd project-07-agent-crew
pip install crewai crewai-tools python-dotenv
```

2 Define your agents

Each agent gets a `role`, a `goal`, and a `backstory` — this shapes how it behaves, similar to a system prompt but scoped to one team member.

`crew.py`

```
from crewai import Agent, Task, Crew, Process
from crewai_tools import SerperDevTool
import os

search_tool = SerperDevTool()  # requires SERPER_API_KEY in .env

researcher = Agent(
    role="Senior Research Analyst",
    goal="Find accurate, current facts about {topic}",
    backstory="You are meticulous and cite sources for every claim.",
    tools=[search_tool],
    verbose=True,
)

writer = Agent(
    role="Content Writer",
    goal="Turn research into an engaging, well-structured article about {topic}",
    backstory="You write clearly for a general audience, without jargon.",
    verbose=True,
```

```
)

editor = Agent(
    role="Managing Editor",
    goal="Polish the article for clarity, tone, and factual consistency",
    backstory="You are a strict but fair editor who catches unclear claims.",
    verbose=True,
)
```

3 Define tasks and how they hand off to each other

The `context` parameter is what makes this a pipeline: it tells the Writer's task to use the Researcher's output, and the Editor's task to use the Writer's output.

```
research_task = Task(
    description="Research the topic '{topic}' and produce a list of key facts with sources.",
    expected_output="A bulleted list of 6-10 factual points, each with a source.",
    agent=researcher,
)

writing_task = Task(
    description="Write a 500-word article about '{topic}' using the research provided.",
    expected_output="A complete draft article with a title and clear structure.",
    agent=writer,
    context=[research_task],
)

editing_task = Task(
    description="Edit the draft for clarity, flow, and consistency with the research facts.",
    expected_output="A final, publish-ready article.",
    agent=editor,
    context=[research_task, writing_task],
)
```

4 Assemble and run the crew

```
crew = Crew(
    agents=[researcher, writer, editor],
    tasks=[research_task, writing_task, editing_task],
    process=Process.sequential,  # each task runs in order
    verbose=True,
)

if __name__ == "__main__":
    topic = input("Article topic: ")
    result = crew.kickoff(inputs={"topic": topic})
    print("\n=== FINAL ARTICLE ===\n")
    print(result)
```

```
python crew.py
```

WATCHING IT THINK

With `verbose=True`, you'll see each agent's reasoning printed live — this transparency is invaluable for debugging multi-agent systems, where it's easy to lose track of which agent produced what.

Sequential vs. hierarchical crews

Process type	How it works	Best for
`Process.sequential`	Tasks run in a fixed order, each with prior context	Predictable pipelines (this project)
`Process.hierarchical`	A manager agent dynamically delegates tasks to workers	Open-ended goals where the plan isn't fixed upfront

Challenge yourself

- ☐ Add a Fact-Checker agent between Writer and Editor that flags unsupported claims
- ☐ Switch to `Process.hierarchical` and add a Manager agent
- ☐ Give the Writer agent a "brand voice" document to follow via a tool

WHAT YOU JUST LEARNED

How to orchestrate multiple specialized agents with clear handoffs — the pattern used in real production systems for content pipelines, customer support triage, and automated research teams.

PROJECT 8 · AGENTIC

AI Coding Assistant Agent

Time	Difficulty	Stack
2–3 hrs	⋆⋆⋆⋆ Advanced	Python, Anthropic tool use, subprocess sandboxing

You'll learn
File-system tools, code execution tools, iterative self-correction

What you're building

An agent that can read files, write code to a file, run it, read the error if it fails, fix the code, and try again — automatically, in a loop — until a script works. This is a simplified version of how tools like Claude Code operate.

If error → agent reads it and fixes the code → runs again. If success → done.

Step-by-step build

1 Set up a sandboxed working directory

Always confine an agent that writes/executes code to a dedicated folder — never let it touch your whole file system.

```
mkdir project-08-coding-agent && cd project-08-coding-agent
mkdir sandbox
```

2 Define file and execution tools

`tools.py`

```
import subprocess, os

SANDBOX = "sandbox"

TOOLS_SCHEMA = [
    {
        "name": "write_file",
        "description": "Write content to a file in the sandbox directory.",
        "input_schema": {
            "type": "object",
            "properties": {
                "filename": {"type": "string"},
                "content": {"type": "string"},
            },
            "required": ["filename", "content"],
        },
    },
    {
        "name": "run_python",
        "description": "Run a Python file in the sandbox and return stdout/stderr.",
        "input_schema": {
            "type": "object",
            "properties": {"filename": {"type": "string"}},
```

```
            "required": ["filename"],
        },
    },
]

def write_file(filename, content):
    path = os.path.join(SANDBOX, filename)
    with open(path, "w") as f:
        f.write(content)
    return f"Wrote {len(content)} characters to {filename}"

def run_python(filename):
    path = os.path.join(SANDBOX, filename)
    result = subprocess.run(
        ["python3", path], capture_output=True, text=True, timeout=10
    )
    if result.returncode == 0:
        return f"SUCCESS.\nSTDOUT:\n{result.stdout}"
    return f"ERROR (exit code {result.returncode}).\nSTDERR:\n{result.stderr}"

def run_tool(name, tool_input):
    if name == "write_file":
        return write_file(**tool_input)
    if name == "run_python":
        return run_python(**tool_input)
    return "Unknown tool"
```

3 Build the self-correcting agent loop

This reuses the exact same loop structure from Project 6 — proof that once you know the pattern, adding new tools is straightforward.

`agent.py`

```
from anthropic import Anthropic
from tools import TOOLS_SCHEMA, run_tool
import os

client = Anthropic(api_key=os.getenv("ANTHROPIC_API_KEY"))
MODEL = "claude-sonnet-4-6"
MAX_ITERATIONS = 8

SYSTEM_PROMPT = (
    "You are a careful coding agent. Write Python code to a file, then run it. "
    "If it errors, read the error carefully, fix the file, and run it again. "
    "Keep iterating until the script runs successfully, then explain what you built."
)

def run_agent(task):
    messages = [{"role": "user", "content": task}]

    for i in range(MAX_ITERATIONS):
        response = client.messages.create(
            model=MODEL,
            max_tokens=1500,
            system=SYSTEM_PROMPT,
            tools=TOOLS_SCHEMA,
            messages=messages,
        )

        if response.stop_reason != "tool_use":
            return "".join(b.text for b in response.content if b.type == "text")

        messages.append({"role": "assistant", "content": response.content})

        tool_results = []
        for block in response.content:
            if block.type == "tool_use":
                print(f"🔧 {block.name}({block.input})")
                result = run_tool(block.name, block.input)
                print(f"   → {result[:200]}")
                tool_results.append({
                    "type": "tool_result",
                    "tool_use_id": block.id,
                    "content": result,
                })
        messages.append({"role": "user", "content": tool_results})

    return "Reached max iterations."
```

4 Try it

```
from agent import run_agent
print(run_agent(
    "Write a script called fib.py that prints the first 15 Fibonacci "
    "numbers, then run it and confirm it works."
))
```

Deliberately ask for something with a likely typo-prone edge case (e.g., "handle negative input by raising a ValueError") and watch the agent hit an error, read it, and self-correct.

Critical safety practices for code-executing agents

ALWAYS SANDBOX EXECUTION

Run agent-generated code in a restricted directory (as shown), and in real projects, inside a container or VM with no network access and a strict timeout — never on your main machine with full permissions.

Risk	Mitigation used in this project
Infinite loop in generated code	`timeout=10` on `subprocess.run`
Writing outside intended folder	All paths are joined under a fixed `SANDBOX` directory
Agent looping forever fixing code	`MAX_ITERATIONS` cap
Destructive commands	Only `write_file` and `run_python` are exposed — no shell/delete tool

Challenge yourself

- ☐ Add a `read_file` tool so the agent can inspect existing code before editing
- ☐ Add a `run_tests` tool that runs pytest and feeds results back
- ☐ Log the full iteration history to review how many attempts each task took

WHAT YOU JUST LEARNED

How to give an agent the ability to write, execute, and iterate on code safely — and why sandboxing and iteration limits aren't optional extras, they're core requirements.

PART 05

Capstone Project

Time to combine everything: conversation memory, RAG over your own documents, tool use, and a proper agent loop — into one cohesive personal AI assistant, built with LangGraph, the industry-standard framework for structuring agent logic as a graph.

PROJECT 9 · CAPSTONE

Personal AI Assistant (Memory + Tools + RAG)

Time	Difficulty	Stack
4–6 hrs	⋆⋆⋆⋆⋆ Capstone	Python, LangGraph, ChromaDB, Streamlit

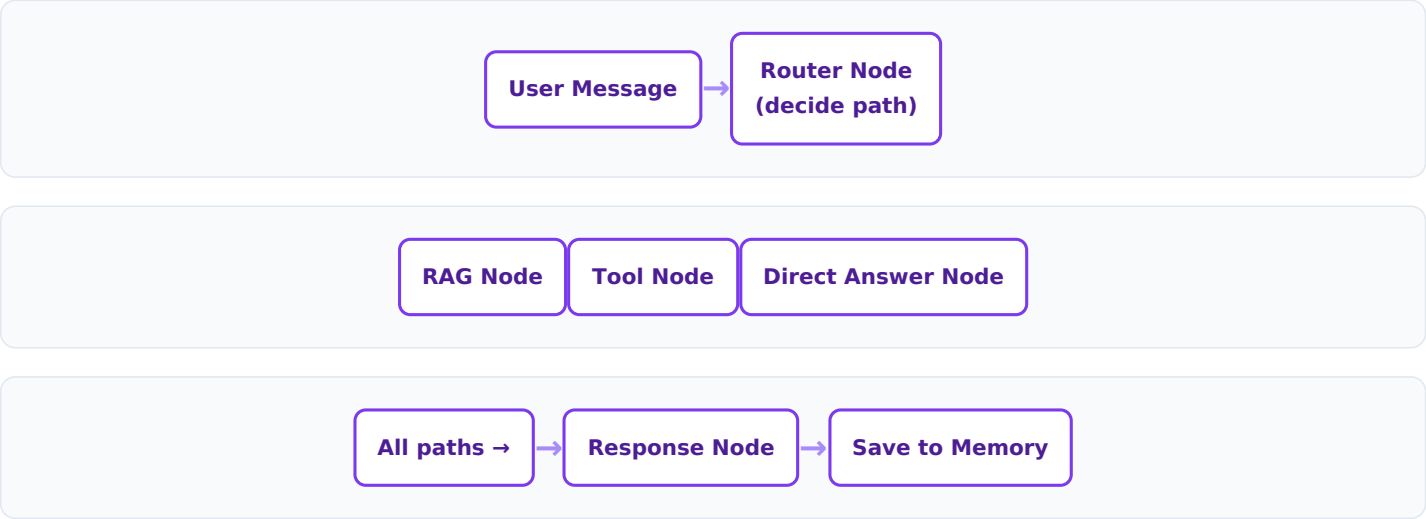
You'll learn
Graph-based agent architecture, combining RAG + tools + memory

What you're building

An assistant that: remembers your conversation across sessions, can search a personal knowledge base of documents you feed it (Project 4's RAG), can use tools like web search and a calculator (Project 6 & 8's tool loop), and decides on its own which of these it needs for any given message.

Why LangGraph instead of a plain loop

Projects 6–8 used a simple `while` loop. That works for one tool-use pattern, but real assistants need **branching logic**: "should I retrieve documents, call a tool, or just answer directly?" LangGraph models your agent as a graph of nodes and edges, making this branching explicit and easy to extend.



Step-by-step build

1 Install dependencies

```
mkdir project-09-personal-assistant && cd project-09-personal-assistant
pip install langgraph langchain-anthropic langchain-community chromadb streamlit python-dotenv
```

2 Define the shared state

In LangGraph, every node reads and writes to one shared `State` object as it flows through the graph.

`state.py`

```
from typing import TypedDict, Annotated
from langgraph.graph.message import add_messages

class AssistantState(TypedDict):
    messages: Annotated[list, add_messages]
    route: str
```

3 Build the router node

This node's only job is to classify the incoming message and pick a path.

`nodes.py`

```
from langchain_anthropic import ChatAnthropic

llm = ChatAnthropic(model="claude-sonnet-4-6")

def router_node(state):
    last_message = state["messages"][-1].content
    classification_prompt = (
        "Classify this user message into exactly one word: "
        "'rag' (asks about the user's own documents/notes), "
        "'tool' (needs current info, web search, or a calculation), "
        "or 'direct' (a normal question you can answer directly).\n\n"
        f"Message: {last_message}\nAnswer with one word only."
    )
    result = llm.invoke(classification_prompt).content.strip().lower()
    route = result if result in ("rag", "tool", "direct") else "direct"
    return {"route": route}
```

4 Build the RAG node (reusing Project 4's logic)

```
from langchain_community.vectorstores import Chroma
from langchain_community.embeddings import HuggingFaceEmbeddings

embeddings = HuggingFaceEmbeddings(model_name="all-MiniLM-L6-v2")
vectordb = Chroma(persist_directory="chroma_db", embedding_function=embeddings)

def rag_node(state):
    question = state["messages"][-1].content
    docs = vectordb.similarity_search(question, k=4)
    context = "\n\n".join(d.page_content for d in docs)
    prompt = f"Answer using this context:\n{context}\n\nQuestion: {question}"
    response = llm.invoke(prompt)
    return {"messages": [response]}
```

5 Build the tool node (reusing Project 6's tools)

```
from tools import web_search  # from Project 6

def tool_node(state):
    question = state["messages"][-1].content
    search_results = web_search(question)
    prompt = f"Using this search info, answer concisely:\n{search_results}\n\nQuestion: {question}"
    response = llm.invoke(prompt)
    return {"messages": [response]}

def direct_node(state):
    response = llm.invoke(state["messages"])
    return {"messages": [response]}
```

6 Wire the graph together

`graph.py`

```
from langgraph.graph import StateGraph, END
from state import AssistantState
from nodes import router_node, rag_node, tool_node, direct_node

def build_graph():
    graph = StateGraph(AssistantState)

    graph.add_node("router", router_node)
    graph.add_node("rag", rag_node)
    graph.add_node("tool", tool_node)
    graph.add_node("direct", direct_node)

    graph.set_entry_point("router")

    graph.add_conditional_edges(
        "router",
        lambda state: state["route"],
        {"rag": "rag", "tool": "tool", "direct": "direct"},
    )

    graph.add_edge("rag", END)
    graph.add_edge("tool", END)
    graph.add_edge("direct", END)

    return graph.compile()
```

7 Add persistent memory across sessions

LangGraph's checkpointer saves state automatically, so the same `thread_id` continues a conversation even after your program restarts.

```
from langgraph.checkpoint.sqlite import SqliteSaver

memory = SqliteSaver.from_conn_string("memory.db")
app = build_graph_with_checkpointer(memory)  # pass checkpointer=memory to .compile()

config = {"configurable": {"thread_id": "user-1"}}
result = app.invoke({"messages": [("user", "Remember my name is Alex")]}, config)
# ... later, even in a new run ...
result = app.invoke({"messages": [("user", "What's my name?")]}, config)  # -> "Alex"
```

8 Build the Streamlit chat interface

app.py

```
import streamlit as st
from graph import build_graph

st.title("🤖 My Personal AI Assistant")

if "app" not in st.session_state:
    st.session_state.app = build_graph()
if "history" not in st.session_state:
    st.session_state.history = []

config = {"configurable": {"thread_id": "streamlit-session"}}

for role, text in st.session_state.history:
    with st.chat_message(role):
        st.write(text)

user_input = st.chat_input("Ask me anything...")
if user_input:
    st.session_state.history.append(("user", user_input))
    with st.chat_message("user"):
        st.write(user_input)

    with st.chat_message("assistant"):
        with st.spinner("Thinking..."):
            result = st.session_state.app.invoke(
                {"messages": [("user", user_input)]}, config
            )
            reply = result["messages"][-1].content
        st.write(reply)
    st.session_state.history.append(("assistant", reply))
```

Extending your capstone into a real product

Extension	What it adds
Document upload in the UI	Users build their own knowledge base, not a preset one
User-specific `thread_id` (login system)	Multiple users, each with private memory
More nodes (e.g., calendar, email draft)	Turns it into a real productivity assistant
Streaming responses	Better perceived speed, matches Project 1's technique
Deploy to Streamlit Community Cloud	A shareable, live link to show in a portfolio

CONGRATULATIONS

You've now built the same architectural building blocks used in real-world AI products: conversational memory, retrieval, tool use, and graph-based orchestration. This capstone is portfolio-ready — write a README, push it to GitHub, and record a short demo video.

PART 06

Where To Go Next

Nine projects in, you now have a working mental model of the entire GenAI-to-Agentic-AI stack. This closing part covers best practices, how to deploy your projects, and where to keep learning.

CHAPTER 10

Best Practices, Deployment & Further Learning

Production checklist for any GenAI project

- ☐ API keys stored in environment variables, never hard-coded or committed
- ☐ A `max_tokens` and iteration limit on every model/agent call
- ☐ Error handling around every API call (rate limits, timeouts, network errors)
- ☐ Input validation before sending user content to the model
- ☐ Logging of prompts and responses for debugging (mind privacy for sensitive data)
- ☐ A billing alert configured on your API provider account

Deploying your projects

Project type	Simple deployment option
Streamlit apps (Projects 2, 3, 4, 5, 9)	Streamlit Community Cloud (free, connects directly to GitHub)
CLI/agent scripts (Projects 1, 6, 8)	Package as a small CLI tool, or wrap in a minimal Flask/FastAPI backend
Multi-agent systems (Project 7)	Run as a scheduled job (e.g., cron, GitHub Actions) or behind a simple API endpoint

Suggested learning path after this book

1 Go deeper on evaluation

Learn to systematically test prompts and agents (not just eyeballing outputs) — look into frameworks like `promptfoo` or building your own eval sets.

2 Learn structured output & guardrails

Explore JSON-schema-constrained outputs and validation libraries like `pydantic` to make agent outputs more reliable.

3 Study production agent architectures

Read how real systems (coding agents, customer-support agents) handle memory, retries, and human-in-the-loop approval.

4 Contribute to open-source

LangChain, LangGraph, and CrewAI are all open source — reading and contributing to their codebases is one of the fastest ways to level up.

> **KEEP BUILDING**
>
> The single best way to get better at this is to keep shipping small projects. Take any idea from your daily life — a habit tracker, a recipe assistant, a study buddy — and rebuild it using the patterns from this book.

YOU MADE IT

Nine Projects.
One Complete Skillset.

From your first API call to a full agentic assistant with memory, retrieval, and tools — you built it all, step by step.

Now go build something of your own.